Exact SAT Solving for Generalized Sudoku: Exactly-One Encodings, SAT Solvers and a Comparison with CP and ILP

Nguyen Thanh Binh[a,*]

[a] *Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay, Hanoi, Vietnam*

ARTICLE INFO

Keywords:
Sudoku
SAT encoding
At-Most-One
Constraint programming
Integer linear programming
PySAT

ABSTRACT

A generalized Sudoku of order N asks to complete an $N \times N$ grid so that every row, column and $\sqrt{N} \times \sqrt{N}$ box holds each value once. We solve it exactly with SAT and study the part of the SAT model that grows fastest: the At-Most-One constraint. On 9×9 to 36×36 puzzles we compare six encodings (Pairwise, Binary, Sequential Counter, Ladder, Product, Commander) under Glucose 3. Commander is fastest up to 25×25, while Pairwise, with 6 times more clauses, is fastest and most stable at 36×36. With Pairwise, five PySAT solvers perform alike up to 36×36; at 49×49 Kissat, CaDiCaL and MiniSat solve all puzzles and Glucose solves 2 of 5. Finally, we compare SAT with CP-SAT, CPLEX CP Optimizer, Gurobi and CPLEX MIP on the same puzzles. CP-SAT is fastest from 25×25 on, SAT with Kissat solves every puzzle at 3 to 4 times its time, and Gurobi solves no puzzle of size 36×36 or larger within 60 s.

1. Introduction

A 9×9 Sudoku is solved by any method in milliseconds. Its generalization to $N \times N$ grids is NP-complete [1], and at $N \geq 36$ the choice of model decides whether a puzzle is solved in a second or not at all. Three exact paradigms apply: Boolean satisfiability (SAT), constraint programming (CP) with the global AllDifferent constraint, and integer linear programming (ILP).

For SAT, the hard part of the model is the Exactly-One constraint: every cell holds one value, and every row, column and box holds each value once. The naive pairwise encoding grows as $\Theta(N^4)$ clauses, so the encoding of the At-Most-One (AMO) half determines the formula size, and also how much unit propagation the solver gets.

This paper makes three contributions:

1. A measured comparison of six AMO encodings (Pairwise, Binary, Sequential Counter, Ladder, Product, Commander) on 9×9 to 36×36 grids, with formula sizes and solve times.
2. A comparison of five SAT solvers available in PySAT [8] (MiniSat, Glucose 3 and 4, CaDiCaL, Kissat) on the best encoding, up to 49×49.
3. A comparison of SAT with CP-SAT, CPLEX CP Optimizer, Gurobi and CPLEX MIP on the same puzzles, machine and time limit.

Section 2 gives the three models, Section 3 the encodings, Section 4 the experiments, and Section 5 the conclusions and limits.

2. Models

A generalized Sudoku of order $N = n^2$ is an $N \times N$ grid split into N boxes of $n \times n$ cells. A solution places one value of $\{1, \ldots, N\}$ in every cell so that every row, column and box contains each value exactly once, and keeps every given clue. Deciding whether a partially filled grid can be completed is NP-complete [1].

*Corresponding author
24021391@vnu.edu.vn (N.T. Binh)
ORCID(s):

Table 1
Exactly-One encodings of one group of m variables: the ALO clause plus the AMO encoding. Clause and auxiliary-variable counts are measured on our implementation at $m = 36$, the group size of a 36×36 grid.

Encoding	Clauses	Aux. vars	Growth	Source	Implementation
Pairwise	631	0	$\Theta(m^2)$	folklore	ours
Binary	217	6	$\Theta(m \log m)$	Frisch et al. [3]	ours
Sequential Counter	105	35	$\Theta(m)$	Sinz [4]	PySAT
Ladder	145	37	$\Theta(m)$	Gent and Nightingale [5]	PySAT
Product	105	22	$\Theta(m)$	Chen [6]	ours
Commander ($k = 3$)	103	16	$\Theta(m)$	Klieber and Kwon [7]	ours

SAT model. The Boolean variable $x_{r,c,v}$ is true when cell (r, c) holds value v, giving N^3 variables. We use the *extended* encoding of Lynce and Ouaknine [2]: an Exactly-One (EO) constraint over each of the $4N^2$ groups below, plus one unit clause $x_{r,c,v}$ per clue.

$$\mathrm{EO}\{x_{r,c,v} : v\} \ \forall r, c, \qquad \mathrm{EO}\{x_{r,c,v} : c\} \ \forall r, v, \tag{1}$$

$$\mathrm{EO}\{x_{r,c,v} : r\} \ \forall c, v, \qquad \mathrm{EO}\{x_{r,c,v} : (r, c) \in B\} \ \forall B, v. \tag{2}$$

Each EO group has exactly N variables. $\mathrm{EO}(X)$ is the At-Least-One (ALO) clause $\bigvee_{x \in X} x$ together with an At-Most-One (AMO) constraint, and the AMO part is where encodings differ (Section 3).

CP model. One integer variable $y_{r,c}$ per empty cell, and an AllDifferent over the empty cells of each row, column and box. A clue is a constant, so its value is removed from the domains of the empty cells that share a row, column or box with it.

ILP model. A 0-1 feasibility model with $x_{r,c,v} \in \{0, 1\}$ for each empty cell and each remaining candidate value v, the constraint $\sum_v x_{r,c,v} = 1$ per cell, and $\sum_{(r,c) \in U} x_{r,c,v} = 1$ for each unit U (row, column or box) and each value v that no clue of U already holds.

The CP and ILP models therefore start from the candidates that remain after the clues are applied. The SAT solver reaches the same reduction itself, because unit propagation of the clue clauses removes those values before the first decision.

3. At-Most-One encodings

Table 1 lists the six AMO encodings we evaluate. The Pairwise encoding needs $\binom{N}{2}$ clauses per group, so the whole grid needs $4N^2\binom{N}{2} = \Theta(N^4)$ clauses: 3.27 million at $N = 36$. The other five add auxiliary variables to reach $O(N \log N)$ or $O(N)$ clauses per group. We describe in detail only the two encodings that win in Section 4: Commander on grids up to 25×25, and Pairwise at 36×36. Binary [3], Sequential Counter [4], Ladder [5] and Product [6] follow their original papers and are only listed.

Pairwise. For every pair x_i, x_j of the group, add $\neg x_i \vee \neg x_j$. It adds no variable, and once one x_i is set true, unit propagation falsifies the other $m - 1$ variables in one step. This is the strongest propagation possible, and the clause count is its only cost.

Commander. The group is split into $\lceil m/k \rceil$ blocks of $k = 3$ variables. Each block G_i gets a new *commander* variable c_i:

$$\mathrm{AMO}_{\mathrm{pairwise}}(G_i) \ \wedge \ \bigwedge_{x \in G_i} (\neg x \vee c_i) \ \wedge \ \mathrm{AMO}(c_1, \ldots, c_{\lceil m/k \rceil}). \tag{3}$$

A true variable raises its commander, and two true variables in different blocks would raise two commanders, which the last term forbids. That term is encoded recursively in the same way, until at most $k + 1$ variables remain and Pairwise finishes. Each level costs $\binom{k}{2} + k = 6$ clauses per block and turns m variables into

Table 2
Median total time (s) of each Exactly-One encoding under Glucose 3, with puzzles solved out of 10. Bold: fastest per size.

Encoding	9×9	16×16	25×25	36×36
Pairwise	0.0048 (10/10)	0.064 (10/10)	0.487 (10/10)	**2.229** (10/10)
Binary	0.0057 (10/10)	0.043 (10/10)	0.222 (10/10)	2.721 (10/10)
Sequential	0.0062 (10/10)	0.032 (10/10)	0.195 (10/10)	9.087 (10/10)
Product	0.0043 (10/10)	0.036 (10/10)	0.161 (10/10)	9.951 (10/10)
Commander	**0.0040** (10/10)	**0.025** (10/10)	**0.143** (10/10)	5.353 (10/10)
Ladder	0.0081 (10/10)	0.053 (10/10)	0.245 (10/10)	5.275 (10/10)

$m/3$, so the total is about $3m$ clauses and $m/3 + m/9 + \cdots \approx m/2$ auxiliary variables. Propagation is as strong as Pairwise: a true $x \in G_i$ forces c_i, the commander AMO falsifies every other c_j, each clause $\neg x' \vee c_j$ then falsifies every variable of the other blocks, and the pairwise clauses of G_i falsify the rest of the block. Commander reaches this with 16 auxiliary variables at $m = 36$, the fewest of the linear encodings. Since both encodings propagate the same, a speed difference between them comes from the size of the formula and from the search, which can now branch on auxiliary variables.

Correctness check. An encoding error either removes valid grids or lets two values share a group. Both change the number of solutions, so our test suite enumerates every solution of the empty 4×4 grid under each encoding (adding a blocking clause over the N^3 primary variables after each model) and requires exactly 288, the known number of 4×4 Sudoku grids. Every encoding passes. Every board returned during the experiments is also checked against the rules and the clues.

4. Experiments

Setup. All runs use one laptop: Intel Core i5-13450HX, 15 GB RAM, Ubuntu 24.04, Python 3.12. SAT solvers are called through PySAT 1.9 [8]. The other solvers are OR-Tools CP-SAT 9.15 [13], Gurobi 13.0 [14] with an academic license, and CPLEX and CP Optimizer 22.2 Community Edition [15]. Every solver runs on one thread, since the SAT solvers are sequential. Runs are sequential, so they do not compete for the CPU.

Instances. A puzzle of order N is generated from a valid complete grid whose bands, stacks, rows, columns and symbols are shuffled at random; then 35% of the cells are kept as clues. Every puzzle is therefore satisfiable. Seeds 0–9 give 10 puzzles per size (5 at 49×49), and every method of an experiment solves the same puzzles.

Measures. The time of a run is wall-clock time from the start of model building in Python to the solver's answer, so it includes encoding. The limit is 60 s per run, and a run that reaches it counts as a timeout (TO). Each table cell gives the median time over the puzzles, counting a timeout as infinite, and in brackets how many puzzles were solved. A median is TO when more than half of the runs timed out. Every returned board is checked, and none was invalid.

4.1. Exactly-One encodings

Table 2 shows two regimes. Up to 25×25, every encoding solves in under 0.5 s and the total time follows the formula size: Commander, the smallest formula (Table 1), is fastest at every size, and Pairwise is slowest at 25×25 with 4 times as many clauses as Commander. At 36×36 the order reverses. Pairwise is fastest with a median of 2.2 s, and all ten of its runs lie between 2.0 and 3.4 s. The linear encodings have medians of 5.3 to 10.0 s and range from 0.8 to 31 s on the same puzzles. Pairwise spends a median 1.2 s building its 3.27 million clauses in Python and only 0.9 s solving, while Commander needs 0.3 s to encode and 5.1 s to solve. Since Section 3 shows the two propagate the same, the gap at 36×36 lies in the search, not in propagation. A plausible cause, which we did not measure, is that the solver branches on auxiliary variables, which carry

Table 3
Median total time (s) of five PySAT solvers on the Pairwise encoding, with puzzles solved (10 per size, 5 at 49×49). Bold: fastest per size.

Solver	9×9	16×16	25×25	36×36	49×49
minisat22	0.0051 (10/10)	0.067 (10/10)	**0.481** (10/10)	**1.966** (10/10)	**10.1** (5/5)
glucose3	0.0049 (10/10)	**0.065** (10/10)	0.492 (10/10)	2.199 (10/10)	TO (2/5)
glucose4	**0.0048** (10/10)	0.065 (10/10)	0.484 (10/10)	2.326 (10/10)	TO (2/5)
cadical195	0.014 (10/10)	0.101 (10/10)	0.674 (10/10)	2.868 (10/10)	11.9 (5/5)
kissat404	0.011 (10/10)	0.099 (10/10)	0.612 (10/10)	2.633 (10/10)	10.3 (5/5)

Table 4
Median total time (s) of SAT (Pairwise encoding) and the CP and ILP solvers, with puzzles solved (10 per size, 5 at 49×49). SL: the CPLEX Community Edition refused the model size. Bold: fastest per size.

Method	9×9	16×16	25×25	36×36	49×49
SAT (glucose3)	**0.0060** (10/10)	0.073 (10/10)	0.533 (10/10)	2.583 (10/10)	TO (2/5)
SAT (kissat404)	0.012 (10/10)	0.114 (10/10)	0.731 (10/10)	2.830 (10/10)	9.584 (5/5)
CP-SAT	0.0079 (10/10)	0.041 (10/10)	**0.138** (10/10)	**0.655** (10/10)	**3.169** (5/5)
CPLEX-CP	0.0096 (10/10)	**0.026** (10/10)	SL	SL	SL
Gurobi	0.0088 (10/10)	0.028 (10/10)	9.006 (10/10)	TO (0/10)	TO (0/5)
CPLEX-MIP	0.020 (10/10)	SL (3/10)	SL	SL	SL

no direct meaning on the board. Once solving dominates, the extra second Pairwise spends on encoding no longer matters. We use Pairwise for the remaining experiments, because they reach 36×36 and beyond.

4.2. SAT solvers

Table 3 compares MiniSat 2.2 [9], Glucose 3.0 and 4.1 [10], CaDiCaL 1.9.5 [11] and Kissat 4.0.4 [12]. Up to 36×36 the five solvers stay within 0.9 s of each other, because Python encoding takes most of the time. CaDiCaL and Kissat are slightly slower on small grids for a technical reason: PySAT cannot interrupt them, so they run in a separate process that is killed at the limit, and starting it costs about 5 to 10 ms. At 49×49, the formula has 11.3 million clauses and takes a median 3.7 s to encode, and the solvers separate: MiniSat, CaDiCaL and Kissat solve all 5 puzzles in a median 10 to 12 s, while both Glucose versions solve only 2.

4.3. SAT against CP and ILP

Table 4 runs SAT with Glucose 3, the baseline of the earlier experiments, and with Kissat, which solved every 49×49 puzzle in Table 3. Up to 16×16 every method answers in under 0.12 s. From 25×25 on, three groups appear:

- **CP-SAT** is fastest and solves every puzzle: 0.14, 0.66 and 3.2 s at 25×25, 36×36 and 49×49. Its AllDifferent propagation works on whole units instead of on the N^3 Boolean variables.
- **SAT with Kissat** also solves every puzzle, 3 to 4 times slower than CP-SAT at the two largest sizes (2.8 and 9.6 s). Part of the gap is Python encoding, which takes a median 3.7 s at 49×49. Glucose 3 solves only 2 of the 5 largest puzzles.
- **Gurobi** slows sharply: 9.0 s at 25×25, and no puzzle solved within 60 s at 36×36 or 49×49.

CPLEX could only be compared on small grids, because the Community Edition limits a MIP model to 1000 variables and constraints and a CP model to a search space of 2^{1000}. CPLEX CP Optimizer was the fastest method at 16×16; CPLEX MIP solved the 3 of 10 16×16 puzzles that fit its limit.

5. Conclusion

On generalized Sudoku up to 49×49, the best AMO encoding depends on the grid size. Commander gives the smallest formula and the fastest runs up to 25×25, while at 36×36 Pairwise is fastest and the most stable, although it has 6 times as many clauses. The choice of SAT solver matters only on the largest grids, where Kissat, CaDiCaL and MiniSat solve all puzzles and Glucose solves 2 of 5. Against other paradigms, CP-SAT is the fastest exact method at every size from 25×25 on, SAT with Kissat follows at 3 to 4 times its time, and Gurobi fails from 36×36.

The study has limits. All puzzles come from one generator with 35% clues, so other clue ratios, which can be much harder, are not covered. Runs use one machine and one thread. CPLEX was limited by its Community Edition. The encoding is built in Python, which takes 26 to 48% of the time of SAT with Kissat at 49×49. Future work: study the effect of the clue ratio, move encoding to compiled code, and test CPLEX with an academic license.

CRediT authorship contribution statement

Nguyen Thanh Binh: Methodology, Software, Validation, Investigation, Writing – original draft.

Declaration of competing interest

The authors declare no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code, the benchmark scripts, the raw results (one CSV row per run) and the scripts that build every table of this paper are available at `https://github.com/Binhbocnc1234/SudokuSAT`, a fork of the first implementation [16].

Declaration of generative AI in the writing process

The authors used Claude (Anthropic) to help write the code and draft the text. Every number in the paper comes from the result files in the repository. The authors reviewed the content and take full responsibility for it.

References

[1] T. Yato, T. Seta, Complexity and completeness of finding another solution and its application to puzzles, IEICE Trans. Fundamentals E86-A (5) (2003) 1052–1060.

[2] I. Lynce, J. Ouaknine, Sudoku as a SAT problem, in: Proc. 9th Int. Symposium on Artificial Intelligence and Mathematics (AIMATH), 2006.

[3] A.M. Frisch, P.A. Giannaros, SAT encodings of the at-most-one constraint: some old, some new, some fast, some slow, in: Proc. 9th Workshop on Constraint Modelling and Reformulation (ModRef), 2010.

[4] C. Sinz, Towards an optimal CNF encoding of Boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005), LNCS 3709, Springer, 2005, pp. 827–831.

[5] I.P. Gent, P. Nightingale, A new encoding of AllDifferent into SAT, in: Proc. 3rd Workshop on Modelling and Reformulating Constraint Satisfaction Problems (ModRef), 2004.

[6] J. Chen, A new SAT encoding of the at-most-one constraint, in: Proc. 9th Workshop on Constraint Modelling and Reformulation (ModRef), 2010.

[7] W. Klieber, G. Kwon, Efficient CNF encoding for selecting 1 from N objects, in: Proc. 4th Workshop on Constraints in Formal Verification (CFV), 2007.

[8] A. Ignatiev, A. Morgado, J. Marques-Silva, PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing (SAT 2018), LNCS 10929, Springer, 2018, pp. 428–437.

[9] N. Eén, N. Sörensson, An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing (SAT 2003), LNCS 2919, Springer, 2004, pp. 502–518.

[10] G. Audemard, L. Simon, Predicting learnt clauses quality in modern SAT solvers, in: Proc. IJCAI 2009, pp. 399–404.
[11] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, F. Pollitt, CaDiCaL 2.0, in: Computer Aided Verification (CAV 2024), LNCS 14681, Springer, 2024, pp. 133–152.
[12] A. Biere, K. Fazekas, M. Fleury, M. Heisinger, CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020, in: Proc. SAT Competition 2020, pp. 51–53.
[13] L. Perron, F. Didier, CP-SAT, Google OR-Tools v9.15, https://developers.google.com/optimization/.
[14] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual, version 13.0, 2026, https://www.gurobi.com.
[15] IBM, ILOG CPLEX Optimization Studio 22.2 (Community Edition), https://www.ibm.com/products/ilog-cplex-optimization-studio.
[16] SudokuSAT: a representation of the Sudoku problem in SAT, GitHub repository, 2026, https://github.com/Vanillinium/SudokuSAT.